

5. Diagramas del Sistema

Los siguientes diagramas están codificados en formato Mermaid, compatible con GitHub, GitLab y editores de código modernos.

1. Arquitectura del Sistema (Despliegue)

```
graph TB
    subgraph "Cliente (Navegador)"
        A[HTML + Bootstrap 5]
        B[Chart.js - Gráficas]
        C[JavaScript - Fetch API]
    end

    subgraph "Servidor Django"
        direction TB
        D[URL Router - urls.py]
        E[Middleware - JWT Auth]
        F[RoleRequiredMiddleware]

        subgraph "Vistas"
            G[TemplateViews - Frontend]
            H[APIViews - REST Endpoints]
        end

        subgraph "Servicios"
            I[ETLEngine - Pandas]
            J[MedicalStatsService]
            K[RiskPredictor - Scikit-learn]
            L[ReportExporter - ReportLab]
        end

        subgraph "Modelos ORM"
            M[CustomUser]
            N[PacienteClinico]
            O[RegistroETL]
        end
    end

    subgraph "Almacenamiento"
        P[(SQLite / PostgreSQL)]
        Q[modelo_entrenado.pkl]
        R[dataset_clinico_etl_1800_registros.xlsx]
    end

    A --> D
    B --> H
    C --> H
    D --> E
    E --> F
    F --> G
    F --> H
    G --> M
    G --> N
    H --> I
    H --> J
    H --> K
    H --> L
    I --> R
```

```
I --> N
I --> O
J --> N
K --> N
K --> Q
L --> N
M --> P
N --> P
O --> P
```

2. Arquitectura Cliente-Servidor (Flujo de Datos)

```
sequenceDiagram
    participant U as Usuario
    participant B as Navegador
    participant S as Servidor Django
    participant DB as Base de Datos
    participant ML as Modelo ML

    Note over U,ML: Inicio de Sesión
    U->>B: Ingresa credenciales
    B->>S: POST /api/auth/login/
    S->>DB: Validar usuario
    DB-->>S: Usuario válido
    S-->>B: Tokens JWT (access + refresh)
    B->>B: Almacena tokens en localStorage

    Note over U,ML: Dashboard
    B->>S: GET /api/analytics/kpis/ (Bearer token)
    S->>DB: Consultar KPIs
    DB-->>S: Resultados
    S-->>B: KPIs en JSON
    B->>B: Renderizar tarjetas y gráficas

    Note over U,ML: Proceso ETL
    U->>B: Clic en "Ejecutar ETL"
    B->>S: POST /api/etl/run/
    S->>S: ETLEngine.extract() - Lee Excel
    S->>S: ETLEngine.transform() - Limpia datos
    S->>DB: ETLEngine.load() - bulk_create
    DB-->>S: Inserción exitosa
    S-->>B: Log ETL con métricas

    Note over U,ML: Predicción ML
    U->>B: Ingresa datos del paciente
    B->>S: POST /api/ml/predecir/
    S->>ML: Cargar modelo .pkl
    ML-->>S: Predicción
    S-->>B: Riesgo + Probabilidad
```

3. Flujo del Proceso ETL (Detallado)

```
flowchart TD
    subgraph "FASE 1: EXTRACT"
        A[Archivo Excel/CSV] --> B{¿Tipo de archivo?}
        B -->|Excel| C[pd.read_excel]
```

```

B -->|CSV| D[pd.read_csv]
C --> E[DataFrame con datos crudos]
D --> E
E --> F[Registrar metadatos: filas, columnas]
end

subgraph "FASE 2: TRANSFORM"
F --> G[1. Normalizar nombres de columnas]
G --> H[2. Eliminar duplicados por id_paciente]
H --> I[3. Convertir edad a numérico + imputar nulos]
I --> J[4. Convertir presiones a numérico + imputar]
J --> K[5. Estandarizar sexo: M/F]
K --> L[6. Normalizar diagnósticos preliminares]
L --> M[7. Remover outliers clínicos]
M --> N[8. Imputar valores nulos restantes]
N --> O[9. Recalcular IMC = peso / altura²]
O --> P[10. Calcular riesgo_enfermedad]

P --> Q{Reglas de Riesgo}
Q -->|Presión > 180 o Glucosa > 300 o Sat < 85| R[Crítico]
Q -->|Presión > 140 o Glucosa > 200 o IMC > 35| S[Alto]
Q -->|IMC > 25 o Fumador o Colesterol > 240| T[Medio]
Q -->|Ninguna condición anterior| U[Bajo]
end

subgraph "FASE 3: LOAD"
R --> V[DataFrame limpio y transformado]
S --> V
T --> V
U --> V
V --> W[Eliminar registros previos en BD]
W --> X[bulk_create - inserción optimizada]
X --> Y[Crear log en RegistroETL]
Y --> Z[Retornar resultado: exitoso/fallido]
end

style R fill:#ff4444,color:white
style S fill:#ff8800,color:white
style T fill:#ffbb00,color:black
style U fill:#00cc66,color:white
style A fill:#1e3a8a,color:white
style Z fill:#1e3a8a,color:white

```

4. Modelo Entidad-Relación (Base de Datos)

```

erDiagram
    CUSTOMUSER ||--o{ REGISTROETL : ejecuta
    CUSTOMUSER {
        int id PK
        string password
        datetime last_login
        boolean is_superuser
        string username UK
        string email
        boolean is_staff
        boolean is_active
        datetime date_joined
        string rol "administrador | medico | analista"
    }
    PACIENTECLINICO {
        int id PK
    }

```

```

    int id_paciente UK
    string nombres
    string apellidos
    int edad
    string sexo "M | F"
    float peso "kg"
    float altura "m"
    float IMC "Calculado: peso/altura^2"
    int presion_sistolica "mmHg"
    int presion_diastolica "mmHg"
    int frecuencia_cardiaca "bpm"
    float glucosa "mg/dL"
    float colesterol "mg/dL"
    float saturacion_oxigeno "porcentaje"
    float temperatura "grados Celsius"
    boolean antecedentes_familiares
    boolean fumador
    boolean consumo_alcohol
    string actividad_fisica "Sedentario | Moderado | Activo"
    string diagnostico_preliminar "Diabetes Tipo 2 | Hipertension | etc"
    string riesgo_enfermedad "Bajo | Medio | Alto | Critico"
    date fecha_consulta
}

REGISTROETL {
    int id PK
    datetime fecha
    int registros_procesados
    int registros_limpios
    int duplicados_eliminados
    float tiempo_ejecucion "segundos"
    string estado "exitoso | fallido"
    string errores "NULL si exitoso"
    bigint usuario_fk_id FK
}

DJANGO_MIGRATIONS {
    int id PK
    string app
    string name
    datetime applied
}

```

5. Flujo de Autenticación JWT

```

sequenceDiagram
    participant U as Usuario
    participant F as Frontend
    participant B as Backend Django
    participant DB as Base de Datos

    U->>F: Ingresa usuario y contraseña
    F->>B: POST /api/auth/login/
    B->>DB: Buscar usuario y validar
    DB-->>B: Usuario encontrado
    B->>B: Generar tokens JWT
    B-->>F: {access_token, refresh_token}
    F->>F: Guardar en localStorage

    Note over U,DB: Navegación protegida
    U->>F: Solicita página protegida
    F->>B: GET / (incluye token en cabecera)

```

```

B->>B: Validar token JWT
B->>B: Verificar rol del usuario
B-->>F: Renderizar página según rol

Note over U,DB: API Request
U->>F: Acción en dashboard
F->>B: GET /api/analytics/kpis/
Note over F: Authorization: Bearer <token>
B->>B: JWTAuthentication valida
B->>DB: Consultar datos
DB-->>B: Resultados
B-->>F: JSON response
F->>F: Renderizar con Chart.js

Note over U,DB: Token expirado
F->>B: Request con token expirado
B-->>F: 401 Unauthorized
F->>B: POST /api/auth/refresh/
Note over F: {refresh: <refresh_token>}
B-->>F: {access: <nuevo_token>}
F->>F: Reintentar request original

```

6. Pipeline de Machine Learning

```

flowchart LR
    A[PacienteClinico BD] -->|Query ORM| B[DataFrame Pandas]
    B --> C[Seleccionar Features]
    C --> D[8 Variables Predictoras]
    D --> E[Train/Test Split 80/20]
    E --> F[Entrenar RandomForest]
    F --> G[Guardar modelo .pkl]
    F --> H[Evaluar métricas]
    H --> I[Accuracy, Precision, Recall, F1]

    G --> J[Cargar modelo]
    K[Datos nuevos paciente] --> J
    J --> L[Predict: clase + probabilidad]
    L --> M[Riesgo: Bajo/Medio/Alto/Critico]

    D -.-> D1[IMC]
    D -.-> D2[Edad]
    D -.-> D3[Glucosa]
    D -.-> D4[Colesterol]
    D -.-> D5[Presión Sistólica]
    D -.-> D6[Presión Diastólica]
    D -.-> D7[Frecuencia Cardíaca]
    D -.-> D8[Fumador]

```